

TEORÍA DE ALGORITMOS
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico 2

Programación Dinámica

4 de mayo de 2026

Jhonny Andres Gomez
111912

Índice

1. Introducción al problema	3
2. Consigna	3
3. Análisis del problema	4
3.1. Ecuación de Recurrencia	4
3.2. Cálculos y resultado de matriz de seguimiento	4
4. Demostración de optimalidad	5
4.1. Formalización del espacio de soluciones	6
4.2. La recurrencia no sobrestima el valor óptimo ($OPT \leq OPT^*$)	6
4.3. El valor optimo es alcanzable ($OPT \geq OPT^*$)	7
4.4. Conclusión de la Demostración	8
5. Análisis de complejidad	8
5.1. Complejidad del algoritmo principal	8
5.2. Complejidad del algoritmo de reconstrucción	9
6. Análisis de variabilidad	10
6.1. Impacto en el tiempo de ejecución	10
6.2. Impacto en la optimalidad	11
6.3. Diferencias de comportamiento entre algoritmos en caso de empates	11
6.4. Consistencia entre la Programación Dinámica y la reconstrucción en empates	11
7. Mediciones	13
7.1. Ajuste por Cuadrados Mínimos	13
7.2. Análisis de resultados	14
8. Conclusiones	14

1. Introducción al problema

Luego de haber analizado a todos los rivales gracias a tu ayuda, Scaloni definió un cronograma de entrenamiento. Tiene definido qué hacer para cada día de acá al mundial que viene, e incluso más. Para hacerlo más simple, para los próximos n días. El entrenamiento del día i demanda una cantidad de esfuerzo e_i , y podemos decir que la ganancia que nos da dicho entrenamiento es igual al esfuerzo. El entrenamiento que corresponde al día i (así como su esfuerzo y ganancia) son inamovibles: el Chiqui Tapia alquiló las herramientas específicas para cada día, y la AFA está muy ocupada organizando el torneo de 2^{30} equipos del año que viene para andar moviendo cosas. Si la cantidad de energía que se tiene para un día i es $j < e_i$, entonces la ganancia que se obtiene en ese caso es justamente j . (si se tiene más energía que e_i , no es que se pueda usar más para tener más ganancia).

A su vez, los jugadores no son máquinas. La cantidad de energía que tienen disponible para cada día va disminuyendo a medida que pasan los entrenamientos. Suponiendo que los entrenamientos empiezan con los jugadores descansados, el primer día luego de dicho descanso los jugadores tienen energía s_1 . El segundo día luego del descanso tienen energía s_2 , etc... Para cada día hay una cantidad de energía, y podemos decir que $s_1 \geq s_2 \geq \dots \geq s_n$. Scaloni puede decidir dejarlos descansar un día, haciendo que la energía “se renueve” (es decir, el próximo entrenamiento lo harían con energía s_1 nuevamente, siguiendo con s_2 , etc...). Obviamente, si descansan, el entrenamiento de ese día no se hace (y no se consigue ninguna ganancia).

Scaloni no sabe bien cómo hacer para definir los días que deba entrenarse y los días que convenga descansar de tal forma de tener la mayor ganancia posible (y tener mayores probabilidades de ganar el mundial que viene), pero Menotti, exponente del juego bonito en Argentina, le recomendó usar Programación Dinámica para resolver este problema. Nos está pidiendo ayuda para poder resolver este problema.

2. Consigna

1. **[Obligatorio]** Hacer un análisis del problema, plantear la ecuación de recurrencia correspondiente y proponer un algoritmo por programación dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de energía disponible desde el último descanso $s_1, s_2, \dots, s_n$, y el esfuerzo/ganancia de cada día e_i , determinar la máxima cantidad de ganancia que se obtiene de los entrenamientos, considerando posibles descansos.
2. **[Obligatorio]** Demostrar que la ecuación de recurrencia planteada en efecto nos permite obtener la solución óptima al problema.
3. **[Obligatorio]** Escribir el algoritmo planteado. Describir y justificar la complejidad de dicho algoritmo. Considerar analizar por separado la complejidad del algoritmo de programación dinámica y la del algoritmo de reconstrucción.
4. **[Obligatorio]** Analizar si (y cómo) afecta la variabilidad de los valores de los esfuerzos (disponibles y necesarios) a los tiempos y optimalidad del algoritmo planteado. Considerar en particular si es posible que haya un comportamiento diferente entre el algoritmo de programación dinámica y el de reconstrucción.
5. **[Opcional]** Realizar ejemplos de ejecución para encontrar soluciones y corroborar lo encontrado. Adicionalmente, el curso proveerá con algunos casos particulares que deben cumplirse su optimalidad también.
6. **[Opcional]** De las pruebas anteriores, hacer también mediciones de tiempos para corroborar la complejidad teórica indicada. Realizar gráficos correspondientes. Generar todo set de datos necesarios para estas pruebas.
7. **[Opcional]** Agregar cualquier conclusión que parezca relevante.

3. Análisis del problema

El problema consiste en analizar cómo Scaloni puede distribuir los entrenamientos desde el día de la concentración hasta el inicio de la copa del mundo. Tenemos n días de entrenamiento planificados. Cada día i tiene un esfuerzo requerido e_i . Nuestro objetivo es maximizar la ganancia total obtenida para obtener más probabilidades de ganar el campeonato.

El primer día de concentración, los jugadores estarán descansados, por lo que cuentan con la mayor energía posible s_1 . Su energía va disminuyendo a medida que acumulen días consecutivos de entrenamiento, es decir $(s_1 \geq s_2 \geq s_3 \cdots \geq s_n)$. Scaloni puede decidir entrenar un día o dar descanso. Al decidir dar descanso en un día i , la ganancia de ese día es 0, pero la energía de los jugadores se reinicia, volviendo a disponer de s_1 para el siguiente día de entrenamiento.

3.1. Ecuación de Recurrencia

Para comprender mejor el problema podemos realizar una analogía con el problema de "Juan el vago". En el cual la única variable que considerábamos era el día laboral. En este caso hay dos variables que tendremos que considerar para obtener la ganancia del día i .

Definimos $OPT(i, j)$ como la máxima ganancia comenzando desde el día i , día en el que comenzamos nuestro plan de entrenamiento y j , que representa el número de día dentro de nuestro bloque de energía (días que llevamos sin descansar).

La decisión en cada paso se divide en dos posibilidades:

- **Entrenar:** Obtenemos una ganancia entre el $\min(e_i, s_j)$ con lo que en el día siguiente estaremos en estado $(i + 1, j + 1)$, ya que habremos acumulado un día más de esfuerzo.
- **Descansar:** La ganancia es 0, pero permite que los jugadores recuperen su energía. El día siguiente estaremos en estado $(i + 1, 1)$, reiniciando el contador de entrenamientos consecutivos.

Formalmente, la ecuación queda expresada de la siguiente manera:

$$OPT(i, j) = \max \begin{cases} \min(e_i, s_j) + OPT(i + 1, j + 1) & \text{Si decidimos entrenar} \\ OPT(i + 1, 1) & \text{Si decidimos descansar} \end{cases} \quad (1)$$

Casos base: Si $i > n$, no quedan más días para entrenar, por lo tanto $OPT(n + 1, j) = 0$ para cualquier valor de j .

Día (i)	Esfuerzo (e_i)	Energía (s_i)
1	3	6
2	7	4
3	2	2
4	5	1

Cuadro 1: Ejemplo numérico de esfuerzo y energía por día.

Si llevamos j días entrenando consecutivamente, la ganancia que obtenemos hoy es exactamente $\min(e_i, s_j)$.

3.2. Cálculos y resultado de matriz de seguimiento

Esfuerzos e_i : $e_1 = 3, e_2 = 7, e_3 = 2, e_4 = 5$.

Energía s_i : $s_1 = 6, s_2 = 4, s_3 = 2, s_4 = 1$.

Caso base: Día 5 ya terminó la preparación. $OPT(5, j) = 0$, para todo j .

Día 4 ($e_4 = 5$):

El valor de descanso siempre es 0, por lo que $OPT(5, 1) = 0$.

$$j = 1 = \max(\min(5, 6) + 0, 0) = \max(5, 0) = 5 = OPT(4, 1) = 5$$

$$j = 2 = \max(\min(5, 4) + 0, 0) = \max(4, 0) = 4 = OPT(4, 2) = 4$$

$$j = 3 = \max(\min(5, 2) + 0, 0) = \max(2, 0) = 2 = OPT(4, 3) = 2$$

$$j = 4 = \max(\min(5, 1) + 0, 0) = \max(1, 0) = 1 = OPT(4, 4) = 1$$

Día 3 ($e_3 = 2$):

El valor de descanso siempre será $OPT(4, 1) = 5$ (calculado en el paso anterior).

$$j = 1 \implies \max(\min(2, 6) + OPT(4, 2), 5) = \max(2 + 4, 5) = 6 \implies OPT(3, 1) = 6$$

$$j = 2 \implies \max(\min(2, 4) + OPT(4, 3), 5) = \max(2 + 2, 5) = 5 \implies OPT(3, 2) = 5$$

$$j = 3 \implies \max(\min(2, 2) + OPT(4, 4), 5) = \max(2 + 1, 5) = 5 \implies OPT(3, 3) = 5$$

Día 2 ($e_2 = 7$):

El valor de descanso siempre será $OPT(3, 1) = 6$.

$$j = 1 \implies \max(\min(7, 6) + OPT(3, 2), 6) = \max(6 + 5, 6) = 11 \implies OPT(2, 1) = 11$$

$$j = 2 \implies \max(\min(7, 4) + OPT(3, 3), 6) = \max(4 + 5, 6) = 9 \implies OPT(2, 2) = 9$$

Día 1 ($e_1 = 3$):

El valor de descanso siempre será $OPT(2, 1) = 11$.

$$j = 1 \implies \max(\min(3, 6) + OPT(2, 2), 11) = \max(3 + 9, 11) = 12 \implies OPT(1, 1) = 12$$

A partir de los cálculos realizados mediante la ecuación de recurrencia obtenemos la matriz resultante. Los guiones (-) representan estados inalcanzables ($j \leq i$).

$i \setminus j$	1	2	3	4
1	12	-	-	-
2	11	9	-	-
3	6	5	5	-
4	5	4	2	1

Cuadro 2: Matriz OPT resultante. Las filas i representan el día actual y las columnas j los entrenamientos consecutivos. El óptimo global es $OPT(1, 1) = 12$.

4. Demostración de optimalidad

Vamos a demostrar que nuestra ecuación de recurrencia es la óptima para nuestro caso. Realizaremos el análisis mediante la definición formal de la instancia y sus restricciones, y desarrollaremos la demostración formal mediante **Inducción Matemática** sobre la estructura de los subproblemas.

Instancia del problema: Llamaremos a la instancia del problema como la tupla (n, e, s) , definida de la siguiente manera:

- $n \in \mathbb{N}$: Representa la cantidad total de días de entrenamiento.
- $e = (e_1, e_2, \dots, e_n)$: Arreglo donde cada $e_i \geq 0$ representa el esfuerzo requerido para el día i .
- $s = (s_1, s_2, \dots, s_n)$: Arreglo de energía disponible según el bloque de días consecutivos entrenando, cumpliendo la restricción: $s_1 \geq s_2 \geq \dots \geq s_n \geq 0$.

4.1. Formalización del espacio de soluciones

Para formalizar, definimos una solución factible desde el día i hasta el día n (asumiendo que en el día i llevamos entrenando j días consecutivos) como una secuencia de decisiones $D = \{d_i, d_{i+1}, \dots, d_n\}$, donde cada decisión $d_k \in \{\text{Entrenar}, \text{Descansar}\}$.

Dada esta secuencia D y nuestro estado inicial (i, j) , podemos expresar la ganancia total obtenida. Si definimos j_k como la serie de entrenamientos acumulada exactamente en el día k en donde cuyo valor se actualiza dependiendo de los descansos tomados previamente, la ganancia total de la estrategia es:

$$G(D, i, j) = \sum_{k=i}^n [d_k = \text{Entrenar}] \cdot \text{mín}(e_k, s_{j_k}) \quad (2)$$

Aquí utilizamos la notación de Iverson, donde el término $[d_k = \text{Entrenar}]$ evalúa a 1 si la condición es verdadera (se entrena ese día) y a 0 si es falsa (se da descanso).

Como establecimos anteriormente, $OPT(i, j)$ representa el valor máximo de $G(D, i, j)$ sobre todas las secuencias de decisiones posibles. Esto fundamenta la ecuación de recurrencia que buscamos demostrar:

$$OPT(i, j) = \text{máx} \begin{cases} \text{mín}(e_i, s_j) + OPT(i+1, j+1) & \text{Si decidimos entrenar} \\ OPT(i+1, 1) & \text{Si decidimos descansar} \end{cases} \quad (3)$$

Para demostrar que nuestra ecuación de recurrencia encuentra la ganancia máxima, utilizaremos un argumento de **Inducción Matemática** sobre el número de días restantes.

Sea $OPT(i, j)$ el valor calculado por nuestra recurrencia y $OPT^*(i, j)$ la ganancia máxima real que se puede obtener desde el día i con una secuencia acumulada j . Queremos demostrar que $OPT(i, j) = OPT^*(i, j)$ para todo estado válido.

4.2. La recurrencia no sobrestima el valor óptimo ($OPT \leq OPT^*$)

En esta primera parte, demostraremos por inducción que el valor devuelto por la recurrencia es siempre menor o igual a la ganancia máxima real. Esto equivale a decir que cualquier valor que calculemos corresponde, al menos, a la ganancia de una estrategia factible.

Caso Base: $i = n + 1$

Cuando llegamos al día $n + 1$, no quedan días disponibles para entrenar. Nuestra recurrencia define $OPT(n + 1, j) = 0$ para todo j . Dado que la única solución posible en este escenario es no realizar ninguna acción, la ganancia real es $OPT^*(n + 1, j) = 0$. Por lo tanto, el caso base se cumple: $OPT(n + 1, j) = OPT^*(n + 1, j) = 0$.

Hipótesis Inductiva:

Supongamos que para todo día posterior $i' > i$ y para cualquier secuencia j' válida, se cumple que $OPT(i', j') \leq OPT^*(i', j')$.

Paso Inductivo:

Consideremos el estado (i, j) . Por definición, nuestra recurrencia toma el máximo entre dos decisiones. Analicemos los dos casos posibles para el valor de $OPT(i, j)$:

1. Si la recurrencia elige **Entrenar**:

$$OPT(i, j) = \text{mín}(e_i, s_j) + OPT(i+1, j+1)$$

Por la hipótesis inductiva, sabemos que $OPT(i+1, j+1) \leq OPT^*(i+1, j+1)$. Entonces:

$$OPT(i, j) \leq \text{mín}(e_i, s_j) + OPT^*(i+1, j+1)$$

El término de la derecha representa la ganancia de una estrategia específica: entrenar hoy y luego seguir el plan óptimo desde mañana. Si el $\min(e_i, s_j) + OPT^*(i+1, j+1)$ es la ganancia de una estrategia factible para el estado (i, j) , por definición de máximo, ninguna estrategia concreta puede superar el $OPT^*(i, j)$. Por lo tanto, $OPT(i, j) \leq OPT^*(i, j)$.

2. Si la recurrencia elige Descansar:

$$OPT(i, j) = OPT(i+1, 1)$$

Nuevamente, por hipótesis inductiva, $OPT(i+1, 1) \leq OPT^*(i+1, 1)$. Dado que descansar hoy y seguir de forma óptima mañana es una opción válida entre todas las posibles para $OPT^*(i, j)$, concluimos que $OPT(i, j) \leq OPT^*(i, j)$.

En ambos casos, el valor calculado por la recurrencia no supera el óptimo real.

4.3. El valor optimo es alcanzable ($OPT \geq OPT^*$)

En esta segunda parte vamos a demostrar que nuestra recurrencia no se queda corta, es decir que siempre existe una estrategia que alcanza el valor óptimo real. Buscamos probar que para todo $i \in \{1, \dots, n+1\}$ y todo $j \in \{1, \dots, n\}$ válido, se cumple:

$$OPT(i, j) \geq OPT^*(i, j) \quad (4)$$

Caso Base: $i = n+1$

Al igual que en el caso anterior, si ya no hay días disponibles, la ganancia es nula.

$$OPT(n+1, j) = 0 = OPT^*(n+1, j)$$

Hipótesis Inductiva:

Supongamos que para todo día posterior $i' > i$ y para cualquier estado válido j' , la relación se cumple: $OPT(i', j') \geq OPT^*(i', j')$.

Paso Inductivo:

Sea $D^* = \{d_i^*, d_{i+1}^*, \dots, d_n^*\}$ una secuencia de decisiones que representa la solución óptima absoluta para el estado (i, j) . Por definición, su ganancia es:

$$G(D^*, i, j) = OPT^*(i, j)$$

La primera decisión de esta solución óptima, d_i^* , es necesariamente una de las siguientes dos opciones:

1. Si D^* decide Entrenar el día i :

La ganancia total de la solución óptima se puede descomponer en la ganancia de hoy más la ganancia del resto de los días:

$$OPT^*(i, j) = \min(e_i, s_j) + G(D^*, i+1, j+1)$$

Dado que D^* es una solución factible, el sufijo de esta secuencia desde el día $i+1$ es una estrategia válida para el estado $(i+1, j+1)$. Su ganancia no puede superar el óptimo global de ese subproblema:

$$G(D^*, i+1, j+1) \leq OPT^*(i+1, j+1)$$

Aplicando nuestra hipótesis inductiva para el día $i+1$:

$$OPT^*(i+1, j+1) \leq OPT(i+1, j+1)$$

Sustituyendo esto en la ecuación original, obtenemos:

$$OPT^*(i, j) \leq \min(e_i, s_j) + OPT(i+1, j+1)$$

Por definición, nuestra recurrencia calcula el máximo entre todas las opciones posibles, por lo que su valor será mayor o igual a la opción específica de entrenar:

$$\min(e_i, s_j) + OPT(i + 1, j + 1) \leq OPT(i, j)$$

Por transitividad, concluimos que $OPT^*(i, j) \leq OPT(i, j)$.

2. Si D^* decide Descansar el día i :

La ganancia en el día i es nula, y el contador de energía se reinicia a 1 para el día siguiente. La ganancia se expresa como:

$$OPT^*(i, j) = 0 + G(D^*, i + 1, 1)$$

Al igual que en el caso anterior, el sufijo de decisiones no puede superar el óptimo global para ese estado:

$$G(D^*, i + 1, 1) \leq OPT^*(i + 1, 1)$$

Aplicando la hipótesis inductiva:

$$OPT^*(i + 1, 1) \leq OPT(i + 1, 1)$$

Sustituyendo y recordando que nuestra recurrencia siempre elige el valor máximo posible, sabemos que:

$$OPT^*(i, j) \leq OPT(i + 1, 1) \leq OPT(i, j)$$

Nuevamente, por transitividad, llegamos a $OPT^*(i, j) \leq OPT(i, j)$.

Para ambos casos lógicamente posibles, hemos comprobado que la recurrencia propuesta arroja un valor mayor o igual al óptimo real.

4.4. Conclusión de la Demostración

A partir del análisis realizado en las dos secciones anteriores, hemos demostrado rigurosamente que:

$$OPT(i, j) \leq OPT^*(i, j) \quad \text{y} \quad OPT(i, j) \geq OPT^*(i, j)$$

Por la propiedad de antisimetría, se concluye que ambos valores son iguales:

$$OPT(i, j) = OPT^*(i, j) \quad \text{para todo } i \in \{1, \dots, n + 1\}, j \in \{1, \dots, n\} \quad (5)$$

Esto demuestra de manera formal que el algoritmo basado en esta ecuación de recurrencia es correcto y encontrará siempre la solución óptima al problema planteado.

5. Análisis de complejidad

5.1. Complejidad del algoritmo principal

```
1
2 def maximizar_esfuerzo(dias, esfuerzos, energias):
3
4     if dias is None or esfuerzos is None or energias is None:
5         return None, None
6
7     if not isinstance(dias, int) or dias <= 0:
8         return None, None
9
10    if not isinstance(esfuerzos, list) or not isinstance(energias, list):
11        return None, None
12
13    if len(esfuerzos) < dias + 1 or len(energias) < dias + 1:
14        return None, None
```



```
15 ganancia = [[0] * (dias + 2) for i in range(dias + 2)]
16
17 for dia in range(dias, 0, -1):
18     for posicion in range(1, dia + 1):
19         ganancia_minima_entrenando = min(esfuerzos[dia], energias[posicion])
20         valor_entrenando = (ganancia_minima_entrenando + ganancia[dia + 1][
21             posicion + 1])
22         valor_descanso = ganancia[dia + 1][1]
23
24         ganancia[dia][posicion] = max(valor_entrenando, valor_descanso)
25
26 ganancia_optima = ganancia[1][1]
27
28 return ganancia_optima, ganancia
```

El algoritmo propuesto mediante tiene una complejidad temporal de $O(n^2)$ y una complejidad espacial de $O(n^2)$.

Podemos determinar la **complejidad temporal**, observando que el algoritmo consiste en dos bucles anidados. El bucle externo itera sobre los días en orden inverso, es decir, $i \in \{n, n - 1, \dots, 1\}$, realizando exactamente n iteraciones. El bucle interno itera sobre la secuencia de días de entrenamiento, $j \in \{1, 2, \dots, i\}$, ejecutando exactamente i iteraciones para cada valor de i .

El número total de subproblemas evaluados está representado por la sumatoria:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad (6)$$

Dentro del bucle interno, el algoritmo realiza un número constante de operaciones: dos accesos a la matriz de resultados previos, una operación de mínimo mín y una operación de máximo máx. Como todas estas operaciones se ejecutan en tiempo $O(1)$, el tiempo total de ejecución es:

$$T(n) = \frac{n(n+1)}{2} \cdot O(1) = O(n^2) \quad (7)$$

Dado que este cálculo es independiente de los valores de entrada y cuenta las iteraciones exactas, podemos afirmar que la complejidad temporal es fuertemente $O(n^2)$.

Para la **complejidad espacial**, analizamos la estructura de datos utilizada para la memorización. Se emplea una matriz de ganancias bidimensional de tamaño $(n + 2) \cdot (n + 2)$. El número total de celdas reservadas en memoria es:

$$(n + 2)^2 = n^2 + 4n + 4 \quad (8)$$

Dado que cada celda almacena un valor, el espacio en memoria crece de forma cuadrática respecto a la entrada, resultando en una complejidad espacial de $O(n^2)$.

5.2. Complejidad del algoritmo de reconstrucción

```
1 def reconstruir_solucion(ganancias, dias, esfuerzos, energias):
2
3     entrenamientos = []
4
5     dia = 1
6     posicion = 1
7
8     while dia <= dias:
9
10         ganancia_minima_entrenando = min(esfuerzos[dia], energias[posicion])
11         valor_entrenamiento = ganancia_minima_entrenando + ganancias[dia + 1][
12             posicion + 1]
```

```
13
14     if valor_entrenamiento >= ganancias[dia + 1][1]:
15         entrenamientos.append("Entrenar")
16         posicion += 1
17     else:
18         entrenamientos.append("Descansar")
19         posicion = 1
20
21     dia += 1
22
23     return entrenamientos
```

El algoritmo diseñado para reconstruir el camino óptimo requiere una complejidad espacial y temporal lineal de $O(n)$.

Respecto a la **complejidad temporal**, el bucle `while` itera día por día evaluando las decisiones desde el día 1 hasta el día n . Esto resulta en exactamente n iteraciones. En cada paso de la reconstrucción, se realiza una comparación en tiempo constante $O(1)$ para determinar si el valor provino de entrenar o descansar. Por lo tanto:

$$T(n) = n \cdot O(1) = O(n) \quad (9)$$

En cuanto a la **complejidad espacial**, la estructura de datos que almacena el plan óptimo final contiene exactamente n elementos una decisión de *Entrenar* o *Descansar* por cada día. Con lo que la memoria adicional requerida por la reconstrucción es $O(n)$.

6. Análisis de variabilidad

En esta sección analizaremos cómo afecta la variabilidad de los valores de energía disponible (s_j) y el esfuerzo requerido e_i al tiempo de ejecución y a la optimalidad del algoritmo propuesto.

6.1. Impacto en el tiempo de ejecución

Recordemos que el número de iteraciones de nuestro doble bucle principal es exactamente:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \quad (10)$$

Esta cantidad de subproblemas no depende de las magnitudes de s_j ni de e_i ; depende únicamente de n . En cada celda evaluada, se ejecuta un conjunto de operaciones de costo constante:

1. $\min(e_i, s_j)$: Comparación entre dos números en tiempo $O(1)$, independientemente de su magnitud.
2. Suma de dos valores extraídos de la tabla en tiempo $O(1)$.
3. $\max$ entre el valor de entrenar y descansar, en tiempo $O(1)$.

Por lo tanto, el tiempo total es:

$$T(n) = \frac{n(n+1)}{2} \cdot O(1) = O(n^2) \quad (11)$$

Concluimos que el tiempo de ejecución es estrictamente independiente de los valores que puedan tomar e_i y s_j .

El **algoritmo de reconstrucción** recorre los días desde 1 hasta n exactamente una vez. El bucle `while` ejecuta siempre n iteraciones, sin importar los valores de e_i , s_j , ni cuántos días de descanso conforman la solución óptima. Por lo tanto, su tiempo es $O(n)$ en cualquier escenario.

6.2. Impacto en la optimalidad

El algoritmo garantiza la optimalidad global sin importar la variabilidad de los datos. Para demostrarlo, analizamos escenarios extremos:

Esfuerzos constantes para todo i : Si todos los días requieren el mismo esfuerzo c , la ganancia de entrenar el día i siendo el j -ésimo de la secuencia es $\min(c, s_j)$. Como $s_1 \geq s_2 \geq \dots \geq s_n$, intercalar un descanso maximiza la energía disponible. La recurrencia lo maneja correctamente: al comparar la opción de entrenar con energía decreciente s_j versus descansar, que reinicia la energía a s_1 , el algoritmo optará por descansar si $s_1 > s_j$, generando un patrón óptimo.

Esfuerzos crecientes: Si los días más valiosos se encuentran al final del cronograma, la optimalidad exige gestionar la energía para llegar con reservas suficientes a esos días críticos. La matriz evalúa el impacto futuro de las decisiones actuales mediante el término $OPT(i+1, j+1)$, asegurando que no se tome una ganancia inmediata, si esta compromete una recompensa mayor en el futuro.

Degeneración por falta de energía ($s_j = 0$): Si la energía decae a 0 para algún día de racha j , entrenar aporta $\min(e_i, 0) = 0$. La recurrencia compara entrenar ($0 + OPT(i+1, j+1)$) contra descansar ($0 + OPT(i+1, 1)$). Dado que siempre se cumple que $OPT(i+1, 1) \geq OPT(i+1, j+1)$ (tener máxima energía nunca es peor), el algoritmo siempre preferirá o empatará eligiendo descansar.

Degeneración por esfuerzo nulo ($e_i = 0$): Si el esfuerzo requerido en un día es 0, la ganancia inmediata de entrenar es $\min(0, s_j) = 0$. Al igual que en el caso anterior, la recurrencia elegirá descansar para reiniciar la energía sin perder ganancias ese día.

6.3. Diferencias de comportamiento entre algoritmos en caso de empates

La variabilidad de los valores e_i y s_j revela una diferencia conceptual clave entre el **algoritmo de Programación Dinámica** y el **algoritmo de reconstrucción**.

El algoritmo calcula el *valor* de la ganancia máxima posible. Si una combinación de e_i y s_j genera un escenario donde la rama de entrenar y la rama de descansar arrojan exactamente la misma ganancia total máxima, existe más de un camino óptimo válido. El algoritmo es indiferente a esta multiplicidad de caminos, ya que en un empate, la función $\max(A, B)$ simplemente retorna el mismo valor, manteniendo la optimalidad intacta.

Sin embargo, el algoritmo de reconstrucción presenta un **comportamiento determinista sesgado**. Al reconstruir el camino, el programa evalúa si el valor almacenado en la tabla provino de la opción de entrenar o de descansar. Ante un empate exacto, la decisión dependerá de cómo se evalúe la condición lógica (por ejemplo, si se prioriza entrenar ante una igualdad). Por lo tanto, utilizando nuestro algoritmo se encuentra la optimalidad absoluta del valor, pero la variabilidad de los datos puede causar que la reconstrucción devuelva solo un camino óptimo específico, pasando desapercibidas otras secuencias de acciones igualmente válidas.

6.4. Consistencia entre la Programación Dinámica y la reconstrucción en empates

Existe una divergencia conceptual en el **caso de empates**, pero la implementación garantiza coherencia.

Cuando la opción de entrenar y la opción de descansar producen exactamente el mismo valor, existe multiplicidad de soluciones óptimas. La Programación Dinámica simplemente almacena el valor máximo acumulado, siendo indiferente al camino. Sin embargo, la reconstrucción está obligada a tomar una decisión determinista para devolver un único plan de acción.

En una celda cualquiera (i, j) , la Programación Dinámica calcula:

$$OPT(i, j) = \max(A, B) \quad (12)$$

Donde:

- $A = \min(e_i, s_j) + OPT(i + 1, j + 1)$ (ganancia de entrenar)
- $B = OPT(i + 1, 1)$ (ganancia si elegimos descansar)

La reconstrucción, en el mismo estado (i, j) , recalcula A utilizando la misma tabla ya construida.

Proposición: Si la reconstrucción utiliza exactamente los mismos valores de la tabla que nuestro algoritmo construye, la solución reconstruida será siempre consistente con el valor óptimo calculado, sin importar la decisión tomada en un empate.

Demostración: La función de reconstrucción accede a $OPT(i + 1, j + 1)$ y a $OPT(i + 1, 1)$, que son exactamente los estados futuros que la Programación Dinámica evaluó para calcular $OPT(i, j)$. Al no haber aleatoriedad ni mutación de la tabla entre ambas fases, los valores A y B recalculados son idénticos a los originales.

En nuestro algoritmo, el criterio de decisión se define mediante la comparación directa: `if (valor_entrenamiento  $\geq$  valor_descanso)`. Ante un empate donde $A = B$, la condición $A \geq B$ resulta verdadera y el algoritmo prioriza la acción de entrenar. Dado que A pertenece al conjunto de los valores máximos, el camino reconstruido garantiza una ganancia predicha por nuestro algoritmo.

En conclusión, aunque existan múltiples caminos óptimos debido a la variabilidad de e_i y s_j , nuestra implementación de reconstrucción actúa de manera determinista hacia el entrenamiento ante los empates, garantizando siempre una solución válida y matemáticamente consistente con el óptimo global hallado.

Ejemplos de ejecución

Se realizaron y analizaron casos de prueba específicos para validar el comportamiento del algoritmo ante diferentes escenarios:

■ Ejemplo A:

- **Datos:** $n = 3$, Esfuerzos: $\{80, 20, 12\}$, Energías: $\{100, 45, 10\}$.
- **Resultado:** 110.
- El algoritmo determina que es preferible entrenar los 3 días seguidos ($80 + 20 + 10$) a pesar de la pérdida de energía ya que en el día 3 la ganancia es $\min(12, 10) = 10$ que esta limitada por la energía y no por el esfuerzo, ya que descansar restaría más ganancia de la que se recuperaría para el día final.

■ Ejemplo B:

- **Datos:** $n = 2$, Esfuerzos: $\{10, 100\}$, Energías: $\{100, 5\}$.
- **Resultado:** 100.
- Aquí el algoritmo detecta que entrenar el día 1 daría solo 10 puntos pero reduciría la energía del día 2 drásticamente. Al elegir *Descansar* el día 1, se llega al día 2 con energía máxima, obteniendo los 100 puntos de ganancia.

■ Ejemplo C Archivo 3.txt de la cátedra:

- **Datos:** $n = 3$, Esfuerzos: $\{1, 5, 4\}$, Energías: $\{10, 2, 2\}$.
- **Resultado:** 7.
- El algoritmo opta por *Descansar* el primer día. Esto permite que en el día 2 donde el esfuerzo es mayor se cuente con la energía máxima disponible, optimizando la ganancia total en comparación a un enfoque que entrene desde el comienzo.

Ejemplo de Ejecución por Consola

A continuación se muestra la salida del programa al procesar los archivos:

```
$ python3 code/main.py ejemplos/1.txt
La ganancia optima es: 110
El plan de entrenamiento es:
Dia 1: Entrenar
Dia 2: Entrenar
Dia 3: Entrenar
```

```
$ python3 code/main.py ejemplos/2.txt
La ganancia optima es: 100
El plan de entrenamiento es:
Dia 1: Descansar
Dia 2: Entrenar
```

```
$ python3 code/main.py ejemplos/3.txt
La ganancia optima es: 7
El plan de entrenamiento es:
Dia 1: Descansar
Dia 2: Entrenar
Dia 3: Entrenar
```

7. Mediciones

Se realizaron mediciones del tiempo de ejecución variando el tamaño de la entrada (n). Los sets de datos se generaron utilizando valores pseudoaleatorios para los esfuerzos y valores ordenados de forma decreciente para las energías, respetando las restricciones del problema.

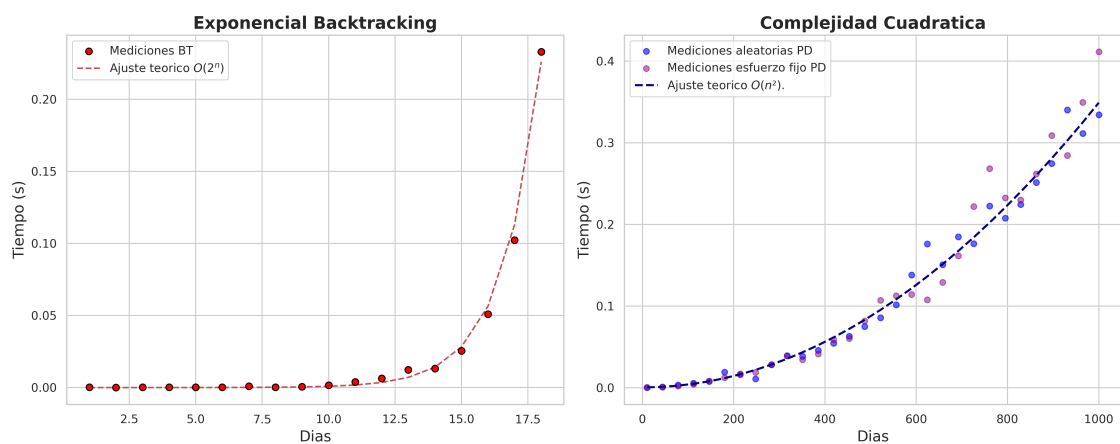


Figura 1: Comparativa de tiempos de ejecución: Backtracking (Exponencial) vs. Programación Dinámica (Cuadrática).

7.1. Ajuste por Cuadrados Mínimos

Los tiempos obtenidos fueron ajustados mediante el método de cuadrados mínimos a las funciones teóricas correspondientes:

1. Para Backtracking: $T(n) = c \cdot 2^n$
2. Para Programación Dinámica: $T(n) = c \cdot n^2$

Como se aprecia en la Figura 1, las mediciones empíricas (puntos) coinciden casi exactamente con los modelos teóricos (líneas punteadas).

7.2. Análisis de resultados

A partir de los gráficos generados, se observa que los resultados validan las hipótesis planteadas:

- **Backtracking:** Presenta un crecimiento exponencial explosivo. A partir de $n \approx 18$, el tiempo de ejecución comienza a ser prohibitivo para el uso cotidiano. Se comprobó experimentalmente que intentar ejecutar esta versión con $n = 50$ resultando en el colapso del sistema por falta de recursos.
- **Programación Dinámica:** El gráfico de la derecha muestra una escala de tiempo drásticamente inferior, con una tendencia cuadrática $O(n^2)$. Mientras que Backtracking sufre para $n = 20$, utilizando Programación Dinámica podemos resolver instancias de $n = 1000$ en aproximadamente 0.1 segundos.

8. Conclusiones

Tras el análisis del problema y el desarrollo de las soluciones, junto con sus mediciones empíricas, se detectaron los siguientes puntos relevantes:

- **Impacto de la técnica de diseño:** Se comprobó que el enfoque de *Backtracking* es inviable para planificaciones de mediano o largo plazo. Mientras que para $n = 18$ el algoritmo responde en segundos, para $n = 25$ el crecimiento exponencial del espacio de soluciones 2^{25} agota los recursos disponibles de la CPU, demostrando que pequeños incrementos en la entrada pueden inutilizar un algoritmo mal escalado.
- **Eficiencia de la Programación Dinámica:** La implementación por Programación Dinámica redujo la complejidad de una clase exponencial a una cuadrática $O(n^2)$. Esto permitió resolver instancias de 1000 días (equivalente a casi tres años de entrenamiento) en menos de 0.1 segundos. Esto confirma que el almacenamiento de subproblemas (*memoization*) es una de las técnicas más potentes para problemas con subestructura óptima y solapamiento de subproblemas.
- **Robustez del Algoritmo:** Se observó que la complejidad de la Programación Dinámica es independiente de los valores de los esfuerzos e_i y energías s_i . A diferencia de otras técnicas que podrían variar su rendimiento según el **peor caso** de los datos, el algoritmo propuesto mantiene un tiempo de ejecución constante para un mismo n , garantizando predictibilidad en sistemas de alta carga de datos.
- **Relevancia del Análisis Empírico:** El ajuste de curvas mediante el método de cuadrados mínimos no solo validó la teoría, sino que permitió identificar el punto exacto de **ruptura** de la técnica de Backtracking. Esto resalta la importancia de realizar pruebas en el desarrollo de software para anticipar comportamientos ante volúmenes de datos reales.

En conclusión, para el problema planteado, la Programación Dinámica no es solo una opción de optimización, sino una necesidad técnica. Sin ella, sería computacionalmente inviable planificar cronogramas extensos asegurando la máxima ganancia de energía y esfuerzo.